

Laboratorio de predicciones

El programa ya está escrito y ya funciona. Lo que se califica no es que corra: es **qué creías tú que iba a imprimir antes de correrlo**, y qué entendiste de las veces que te equivocaste. Es la práctica de tipos de datos y expresiones con operadores, hecha al revés.

En una línea

Corres `Laboratorio.java` y el programa te pregunta, una por una, qué crees que imprime cada línea de código. Contestas las 36, él te dice cuáles fallaste, escribes las cinco líneas del bloque 7, y entregas una **captura de tu tabla final** más un **documento explicando por qué te equivocaste**.

Por qué esta práctica es así

Copiar la salida de la pantalla no cuesta nada y no enseña nada. Predecir sí: en el momento en que escribes "7 / 2 da 3.5" y la máquina te contesta `3`, aprendiste algo que ya no se te va a olvidar en el examen ni en la Tarea 1. **Equivocarse aquí no baja la calificación**. Lo que la baja es no haber predicho, o no poder explicar el fallo.

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	Viernes 11 de septiembre , 23:59. Se empieza en el laboratorio del miércoles y ahí mismo se resuelven las dudas.

Dónde	Su rama <code>entregas_apellido_nombre</code> · carpeta <code>entregas/apellido_nombre/p03/</code> . Solo push. No se abre pull request.
Qué archivos	<code>Laboratorio.java</code> (con tu bloque 7 escrito), <code>Laboratorio.class</code> y tu documento con la captura y el análisis . Tres archivos.
Cómo se califica	Las 36 predicciones contestadas 30 % · el análisis de tus fallos 30 % · bloque 7 corriendo 20 % · captura y documento bien armados 20 %. No se califica cuántas acertaste , sino que expliques las que fallaste.

Lo único que puede sacarte del 100

Entregar la captura sin el análisis. El programa ya te dice cuáles fallaste: lo que se califica es que expliques **por qué**. Nadie acierta las 36 la primera vez, y no se espera que lo hagan. **Ocho o diez fallos bien explicados es una entrega excelente**; 36 aciertos sin una sola explicación, no.

Parte 1 · El orden exacto de trabajo

El programa te va preguntando. No llenas tablas a mano: **tecleas tu predicción y le das Enter**, y él te va diciendo si le atinaste.

1 Compila y corre

`javac Laboratorio.java` y luego `java Laboratorio` . **Córrelo desde la terminal**, no con el botón de "Run" del editor: necesita teclado para preguntarte. Si lo corres sin teclado te avisa y no te sirve de nada.

2 Contesta las 36 preguntas

Por cada línea de código te pregunta qué crees que va a imprimir. Escribe el valor **como se vería en pantalla**: `3` , `3.5` , `true` , `A` , `null` , `Infinity` . Si no tienes idea, **adivina**: una predicción equivocada vale, una en blanco no. Al terminar cada bloque te muestra una tabla con lo que predijiste, lo que salió, y cuáles fallaste.

3 El bloque 7: ahora el código es tuyo

Cinco líneas comentadas que tienes que escribir y descomentar. Cada una con un comentario corto diciendo qué operador usaste y por qué. No vale escribir el resultado a mano: tiene que ser una operación que la máquina calcule. Cuando las tengas, **vuelve a correr el programa**.

3.1 Truco para probar tu bloque 7

Cuando ya escribiste el bloque 7 y solo quieres ver si corre, no tienes que volver a contestar las 36 en serio: **teclea cualquier cosa**, por ejemplo una `x`, y dale Enter rápido hasta llegar al final. Tu `mis-resultados.txt` va a marcar 0 aciertos, y **eso no importa**: esa corrida es solo para probar tu código. La que entregas es la del documento, con tus predicciones de verdad.

4 Captura la tabla final

Al final imprime **tu tabla completa** con las 36, cuántas acertaste, y la lista de las que fallaste. **Tómale captura de pantalla a esa tabla**. El programa además la deja escrita en `mis-resultados.txt`, en la misma carpeta, por si la captura te sale cortada.

5 Escribe tu análisis

Un documento (Word, PDF, lo que uses) con dos cosas: **la captura** de tu tabla final, y **por qué te equivocaste** en cada una que fallaste. Una explicación por fallo, con tus palabras. El programa te deja la lista lista para que la copies.

Parte 2 · Qué hay en cada bloque

BLOQUE	DE QUÉ VA	RENGLONES
1	Con qué nace un atributo al que nadie le puso valor	5
2	La división que miente: entera contra decimal, y qué pasa al dividir entre cero	6

3	El módulo <code>%</code> , incluyendo con negativos y con decimales	6
4	El casting y lo que se pierde: decimales, letras y números que no caben	7
5	Relacionales y lógicos, y por qué una división entre cero a veces no truena	6
6	Las sorpresas: <code>0.1 + 0.2</code> , el entero que se da la vuelta, y el <code>+</code> que pega texto	6
7	Tuyo: cinco expresiones que tienes que escribir	5

Las que casi nadie acierta, para que no te asustes

Hay siete renglones diseñados para que falles: `(double) (7 / 2)`, `7.0 / 0`, `-7 % 2`, `(int) -3.9`, `(byte) 200`, `0.1 + 0.2 == 0.3`, y el par `1 + 2 + "3"` contra `"1" + 2 + 3`. Si fallas esos siete y los explicas bien, tu entrega está mejor que una donde nadie falló nada.

Parte 3 · Cómo se ve una respuesta que sí cuenta

El renglón `(int) -3.9`. Casi todos predicen `-4` y sale `-3`.

Explicación que no cuenta

"Me equivoqué porque pensé que era -4 pero es -3."

✓ Explicación que sí cuenta

"Pensé que el casting redondeaba, y no: corta la parte decimal y ya. Como corta hacia el cero, `-3.9` se queda en `-3`, no en `-4`. Si de verdad quisiera el `-4` tendría que usar otra cosa, no un casting."

La diferencia es que la segunda dice **qué creías que hacía el lenguaje y qué hace en realidad**. Ese es el ejercicio completo.

Parte 4 • Cómo se entrega

```
entregas/  
└─ apellido_nombre/      # TU carpeta, con tu apellido y tu nombre  
    └─ p03/              # las practicas van aqui, directo en tu carpeta  
        ├── Laboratorio.java    # con tu bloque 7 escrito  
        ├── Laboratorio.class  
        └─ P3-analisis.docx     # tu captura + por que fallaste  
└─ tareas/               # las tareas van aparte, dentro de tareas/  
    └─ t01/              # la tarea del lunes
```

```
# 1. parate en TU rama (la misma de todo el semestre)  
git checkout entregas_apellido_nombre  
git pull  
  
# 2. la carpeta de esta practica  
mkdir -p entregas/apellido_nombre/p03  
  
# 3. compila y corre una ultima vez antes de subir  
javac Laboratorio.java  
java Laboratorio  
  
# 4. guarda y sube  
git add entregas/apellido_nombre/p03      # los 3: .java, .class y tu documento  
git commit -m "P3: laboratorio de predicciones"  
git push -u origin entregas_apellido_nombre
```

✓ Checklist antes del último push

☐ Contesté las 36 preguntas, ninguna en blanco ☐ Las cinco líneas del bloque 7 están escritas, descomentadas y corriendo ☐ Escribí mi nombre hasta arriba del archivo ☐ Le tomé captura a la tabla final, y se lee ☐ Mi documento explica **cada** fallo con mis palabras, no copiadas ☐ Subí los tres archivos: `.java`, `.class` y el documento ☐ `git branch` muestra el asterisco en mi rama, no en `main` ☐ Hice `push` y lo verifiqué abriendo mi rama en GitHub

Si algo falla

SÍNTOMA

`class Laboratorio is public, should be declared in a file named Laboratorio.java`

El programa corre pero el bloque 7 no imprime nada

`Exception in thread "main"
java.lang.ArithmeticException: / by zero`

`error: ';' expected` en el bloque 7

`incompatible types: possible lossy conversion from double to int`

`Error: Could not find or load main class Laboratorio.class`

QUÉ PASÓ

Le cambiaste el nombre al archivo. Este no se renombra: se queda `Laboratorio.java`.

Sus cinco líneas siguen comentadas. Quítales el `//` de adelante.

En tu 7.5 usaste `&` en vez de `&&`, o pusiste la división del lado izquierdo. El corto circuito solo protege lo que está a la **derecha** de un `&&`.

Descomentaste la línea pero dejaste los tres puntos `...` adentro del paréntesis. Ahí va tu expresión.

Le estás dando un decimal a un `int` sin casting. Es justo lo del bloque 4.

Corriste `java Laboratorio.class`. Es `java Laboratorio`, sin el `.class`.

Una cosa más

Esta práctica y la Tarea 1 son cosas distintas y van en carpetas distintas. La práctica es de aquí al viernes; la tarea es del lunes 14. Los siete renglones que fallaste en esta práctica son exactamente los errores que te van a salir en la tarea, así que hacerla primero te ahorra tiempo después.